
Current Topics in Digital Philology —

Assignment 4: NLP for Historical Text

Daan Brugmans
Radboud University
daan.brugmans@ru.nl

1 Subtask 1: Old Bailey

1.1 1

1. In *The Proceedings of the Old Bailey of 6th January 1831* (<https://www.oldbaileyonline.org/record/18310106>), on page 135, Edward Williams had their belongings stolen by Thomas Adams. In this passage, it is stated that Edward and Mr. Charles Dalby, who are in a partnership, “[...] are woollen warehousemen”. Edward states, regarding the woollen cloth stolen by Thomas Adams, that “[he] had seen this piece of cloth two or three days previous to its being taken - it was in the inner warehouse;”. Edward also states that the rent of the house he lives in “[...] is paid by the firm”, which would imply that he earns a living by being a woollen warehouseman.
2. In the same case, William Hile states that “[they are] a constable of Bassishaw ward”. He then describes how his role as constable is relevant to the case at hand: “[...] I was passing by the gate of the prosecutors’ premises; (there is a yard leading to their premises) - I saw the prisoner and another coming along the yard- they were inside the yard, coming from the warehouse; the other had the cloth on his shoulder [...]”. Although William does not explicitly state that he earns an income from his role as a constable, the statement “I am a constable of Bassishaw ward” does seem to imply that he is compensated for his work.
3. Also within the same case, Isaac Mander states that “[they are] book-keeper in a broker’s counting-house”. As they do the book-keeping for a different person, a broker, in that different person’s counting-house, it seems like Isaac has a job as a book-keeper and earns a living this way.

1.2 2

1. When searching for all records where the offense was “concealing a birth”, we get records dating from the years 1836 to 1913, thus “concealing a birth” was a crime attested in the period 1836 - 1913.
2. Overall, a lot of cases of “concealing a birth” are found not guilty. Of those who are found guilty, early records often show a punishment of confinement for several months, often 3, 4, or 6 months, but sometimes up to a year. In later records, the punishment is often either a brief imprisonment of a few days or weeks, or of “hard labor”, which can last weeks to a few months.
3. The fact that concealing a birth was considered a crime would imply that at least certain levels of government found the birth of a child important enough to be something that should be registered. However, given the amount of times defendants of the crime are considered not guilty by the court, it would seem that the crime isn’t considered severe, especially in later times where punishments would consist of imprisonment of a few days or even a single day. One reason why concealing a birth might have been a crime is due to economical laws,

such as tax laws, which might have depended on the size of a family, or laws regarding population size and control.

1.3 3

For all records where the punishment was any form of death, 8259 of the records were of male victims and 1857 were of female victims. This immediately shows that more men received a death penalty than women (women have about $\frac{1}{4}$ th of the records men have with a death penalty). I would argue that this is likely due to the fact that men more often take extreme decisions/risks than women, more often committing more severe crimes with more severe punishments, and thus more often being charged for a severe crime and more often sentenced to death.

Among the subcategories of the death penalty offered by the Old Bailey, I could not find notable differences in amounts of verdicts where the distribution of verdicts for men vs. women does not reflect the balance of amount of death sentences in general given to men vs. women. In other words: men were more often punished with a certain type of death sentence than women since they were more often sentenced to death in general, with the number of men being sentenced to a subtype of the death penalty often being around 3 to 4 times that of women. From these findings, I would then conclude that the death penalty does not seem to be given much differently between men and women.

1.4 4

I chose to analyze the crime of embezzlement. Among women found guilty of embezzlement, the victims were often acquitted or found not guilty, and of those who were, punishments were most often imprisonment, ranging from 10 days to 1 year, but most often a few months. For men found guilty of embezzlement, of which the number of records is much larger, victims were also often found not guilty, and of those who were, punishment was most often imprisonment lasting from a week to a year, although there seem to be more cases of imprisonment durations at the higher end of that range for men than women (many embezzlement imprisonments last 9 months or 1 year). A punishment given out later to men is five years of penal servitude, meaning that the victims had to perform forced labor during imprisonment.

I would say that on the basis of these findings, I would conclude that the range of punishments for the crime of embezzlement is very similar between men and women. However, it does seem that men more often received a longer imprisonment time than women, although this may be explained by men more often committing more severe cases of embezzlement.

1.5 5

I want to find out during which period the punishment of death by burning was given, or rephrased into a research question: "During which time period is the punishment of death by burning attested in the records?". I used the Statistical search and searched for all records where the punishment was labeled as "Death > Burning". This resulted in 36 records ranging from 1674 to 1790, giving us the period we wanted. It is notable that the punishment was given out very infrequently, with often plenty of years passing between cases.

1.6 6

1. (a) The documents must first be digitalized before they can be searched. Most preferably, this is done by hand by experts, but assuming that is not an option and that it must be done using just digital technology, we could take images of the pages of the records and have their textual contents parsed by an Optical Character Recognition system.
- (b) Once we have the raw digital text, we can perform a form of preprocessing to format it into a more readable format while maintaining the text's meaning (much easier said than done!)
- (c) We can then perform tokenization on the text to standardize it further, making it more reliably browseable.
- (d) We perform Part-of-Speech tagging on the text so that we know the morphological role of the text's tokens.

- (e) We parse the text so that we can perform a syntactic analysis on it, giving us the role of every token in the text from a syntactic point of view.
 - (f) We can then extract verb phrases using the syntactic analysis to find phrases that are likely to describe any act of work.
 - (g) We store these texts and their verb phrase markings in an information retrieval system.
 - (h) We implement an interface where the information retrieval system can be queried for results.
2. For this pipeline to work, we must annotate the digitalized texts with Parts-of-Speech and syntactic information. These are used to extract the verb phrases that indicate information about work.
 3. Since we want to be able to search by gender, societal role, and historical period, we should annotate texts and their verb phrases with these three pieces of metadata. This is very preferably work that should be done by experts, or at least by humans, as it could be very difficult for automated systems to collect this metadata from the text.

2 Subtask 2: Spelling Normalization

2.1 1

I used NLTK's current default PoS Tagger for this question, the "averaged_perceptron_tagger_eng".

1. (a) In the fifth sentence, the NLTK tagger tags the word "kyng" as a foreign word (FW), while it is not actually a foreign word within the sentence, just a historical spelling for "king". The cause for this wrong tag seems straightforward: the tagger assumes the word isn't an English word, but a word from a different language (although one could argue that Middle English/Early Modern English is a different language from Modern English).
 - (b) In the 21st sentence, the reverse occurs, where "Anglicis" is marked as a proper noun (NNP), while it is actually a foreign word, specifically a Latin form of the word for "English". This mistake is caused by the fact that the phrase that the word is a part of "In Anglicis", is actually a Latin phrase, while the tagger assumed it wasn't. I would consider this occurrence as caused by the nature of the historical text, since code-switching to Latin was much more commonplace in official written documents during that period.
 - (c) In the 26th sentence, the word "hegh" is marked as a noun (NN), while it is actually an adjective. The word is used in the phrase "[...] in hegh whirshypp and easse [...] / [...]] in high worship and ease [...]", which is why I would argue for its role as an adjective. The historical spelling likely confuses the tagger into thinking that "hegh" is a noun, potentially with a missing comma after it.
2. The word "kyng" not being tagged as a noun could have a significant impact in the syntactic analysis of sentence 5, as it is the subject of the sentence, and in information extraction systems the sentence may not be recognized as being about kings. The foreign word "Anglicis" being recognized as a proper noun could confuse named entity recognition systems into thinking that Anglicis is potentially a person or place, while it is actually part of a Latin phrase, making NER results messier. Although "hegh" being labeled as a noun may not be very impactful in this context, it still has the potential to mess up a syntactic analysis of the sentence and its syntax tree.

2.2 2

1. I chose to divide the 500 sentences into a roughly 40-60 train-test split, using the first 200 sentences to come up with 10 rules. I chose this division because I did not want to use a sizeable majority of the text to come up with rules, and I found that I needed 200 sentences to come to 10 rules that I thought were reliable enough to be used for the rest of the dataset.
2. (a) Change the word "ye" to "you", as "ye" occurs frequently in the text and is most often used for that purpose.

- (b) Change words that end in “-bre” to “-ber”, such as the names of months. This historical spelling is sometimes used in the text instead of the modern one.
 - (c) Change the letter thorne (Þ) into “th”. The thorne is an old letter and has fallen out of use. It used to represent “th” and replacing it results in modern spellings of old words.
 - (d) Change any instance of “y” followed by a consonant into “i” followed by a consonant. “y(consonant)” is used a lot as a historical spelling in the texts where nowadays “i(consonant)” is used. To prevent unwanted changes, the “y” isn’t changed when it is followed by a vowel or the end of the word.
 - (e) “ffor” is changed to “for”, as it is seemingly a historical spelling for the modern word going by the text.
 - (f) “vn-” is replaced by “un-” at the start of a word. Historically, “v” was often used instead of “u” in writing. This rule updates this historical spelling to a modern one.
 - (g) “wn-” is replaced by “un-” at the start of a word. The same as “vn-”, but with “wn-” instead.
 - (h) “sch” is replaced by “sh”. “sch” is a spelling still used in other Germanic languages for the sound now written as “sh” in English, and was still being used in the older English of the text. It is thus modernized with this rule.
 - (i) “-eth” is replaced by “-e” at the end of a word. “-eth” was often used as a verb conjugation, but has fallen out of use. The modern “-e” is used instead after the rule is applied. Worth noting is that I think this is the rule that is most likely to match false positives.
 - (j) “-ie” is replaced by “-y” at the end of a word. This is seemingly a historical spelling in words such as “Normandie”, which is nowadays spelled “Normandy”, alongside varying other words.
3. To my understanding, none of the rules are dependant upon one another, neither intentionally nor unintentionally. The result of running any of the rules before another one should not change the resulting text, as none of the patterns for the rule inputs match any of the patterns for the rule outputs.

2.3 3

1. Rule-based spelling normalization is a relatively straightforward normalization method where spelling is normalized using explicit rules, such as those described above. The main advantage of this method is that it is not data-hungry at all and can be used even for very small datasets. However, they require that a human, and preferably an expert, sets up these rules, which can be very time-intensive. Furthermore, the hardness of rules means that there is a big chance that wrongful conversions are made through false positives, with the remedy for this being making a big list of very specific rules, which is also very time-intensive.
2. Levenshtein-based normalization applies normalization by comparing the edit distances between the historical and modern spellings of a word. It has a bunch of advantages: it is language-independent, it doesn’t require any manual work by humans, and we don’t need any data for training a model to do the normalization. However, it does require that there exists a robust corpus of the modern version of the language in which text is normalized, which is not always the case. When such a robust enough dictionary doesn’t exist, Levenshtein-based normalization isn’t feasible.

2.4 4

1. According to my evaluation script, 74.5% of the tokens share the same spelling between the original, unaltered historical text and the ground truth modern English text.
2. According to my evaluation script, 74.8% of the tokens share the same spelling between the historical text after the rules have been applied and the ground truth modern English text.
3. My rules have introduced only very slight improvements. I feel that the base accuracy is already quite high, and when inspecting my implementation during debugging, it seems that the score is achieved by many words already being the same across both the historical and modern variants. As I expected, there were some incorrect changes made by my rules caused by false positives:

- (a) In the sentence starting with “It was in the Elizabeth Bonaventure [...]”, “Elizabeth” was changed to “Elizabe” due to my “-eth” rule. This is simply not correct. One way this may have been avoided is by not applying the rule to proper nouns.
- (b) In the sentence starting with “the K. of Poland is in Silesia hunts and passeth [...]”, “passeth” is changed to “passe”. This is how I expected the implementation of my rule to act, but it is not correct English and should be “passes”. This could’ve been avoided by making the “-eth” rule more specific, or by instead using a check that looks at which verb conjugation would match the noun preceding it.
- (c) In the sentence “I know you will not think it fit my Lord should discharge an Officer of the Field but in a regulate way”, “discharge” is changed to “disharge” due to my “sch” rule. This is unintentional and an edge case I did not consider. One way this could’ve been avoided by checking if the word containing the “sch” is actually a compound word, which is true in this case with “dis-” and “charge”.

A 10 Rules Script

NOTE: since the letter thorne (Þ) cannot be displayed properly in the code block, it is indicated using “(THORNE)”.

```
import re
from pathlib import Path

import nltk

def print_nltk_pos_tags():
    with open(path_to_hs_file, "r", encoding="utf-8") as hs_file:
        hs_file_contents = hs_file.readlines()

        for i, sentence in enumerate(hs_file_contents):
            print(i)
            print(" ", nltk.tag.pos_tag(nltk.tokenize.word_tokenize(sentence)))

if __name__ == "__main__":
    path_to_assignment_4 = Path(__file__).parents[0]
    path_to_hs_file = Path(path_to_assignment_4, "icamet-samples_hs.txt")
    path_to_en_file = Path(path_to_assignment_4, "icamet-samples_en.txt")

    normalization_rules = {
        r"\syē\s": r" you ",
        r"bre\s": r"ber ",
        r"bre$": r"ber ",
        r"(THORNE)": r"th",
        r"y([bcdfghjknmlnpqrstvwxyz])": r"i\l",
        r"ffor": r"for",
        r"\swn": r" un",
        r"\svn": r" un",
        r"sch": r"sh",
        r"eth\s": r"e ",
        r"ie\s": r"y ",
    }

    with open(path_to_hs_file, "r", encoding="utf-8") as hs_file:
        hs_file_contents = hs_file.readlines()

    hs_file_contents_rules_applied = []
    for sentence in hs_file_contents[200:]:
        for regex, substitution in normalization_rules.items():
            sentence = re.sub(regex, substitution, sentence)

        hs_file_contents_rules_applied.append(sentence)

    with open(
        Path(path_to_assignment_4, "icamet-samples_hs_rules.txt"), "w", encoding="utf-8"
    ) as hs_file_with_rules:
        hs_file_with_rules.write("".join(hs_file_contents_rules_applied))
```

B Evaluation Script

NOTE: since the letter thorne (Þ) cannot be displayed properly in the code block, it is indicated using “(THORNE)”.

```
import re
from pathlib import Path

import nltk

def print_nltk_pos_tags():
    with open(path_to_hs_file, "r", encoding="utf-8") as hs_file:
        hs_file_contents = hs_file.readlines()

        for i, sentence in enumerate(hs_file_contents):
            print(i)
            print(" ", nltk.tag.pos_tag(nltk.tokenize.word_tokenize(sentence)))

def apply_normalization_rules():
    normalization_rules = {
        r"\syе\s": r" you ",
        r"bre\s": r"ber ",
        r"bre$": r"ber ",
        r"(THORNE)": r"th",
        r"y([bcdfghjknmlnpqrstvwxyz])": r"i\1",
        r"ffor": r"for",
        r"\swн": r" un",
        r"\svн": r" un",
        r"sch": r"sh",
        r"eth\s": r"e ",
        r"ie\s": r"y ",
    }

    with open(path_to_hs_file, "r", encoding="utf-8") as hs_file:
        hs_file_contents = hs_file.readlines()

        hs_file_contents_rules_applied = []
        for sentence in hs_file_contents[200:]:
            for regex, substitution in normalization_rules.items():
                sentence = re.sub(regex, substitution, sentence)

            hs_file_contents_rules_applied.append(sentence)

        with open(
            Path(path_to_assignment_4, "icamet-samples_hs_rules.txt"), "w", encoding="utf-8"
        ) as hs_file_with_rules:
            hs_file_with_rules.write("\n".join(hs_file_contents_rules_applied))

def get_normalization_accuracy(
    historical_text: list[str], ground_truth: list[str]
) -> float:
    if len(historical_text) != len(ground_truth):
        raise ValueError(
            "Number of sentences in the historical text and the ground truth text sh
        )

    tokenized_historical_text = []
```

```

for sentence in historical_text:
    tokenized_historical_text.append(nltk.tokenize.word_tokenize(sentence))

tokenized_ground_truth = []
for sentence in ground_truth:
    tokenized_ground_truth.append(nltk.tokenize.word_tokenize(sentence))

total_token_count = 0
matching_token_count = 0
for historical_sentence, ground_truth_sentence in zip(
    tokenized_historical_text, tokenized_ground_truth
):
    for historical_token, ground_truth_token in zip(
        historical_sentence, ground_truth_sentence
    ):
        if historical_token == ground_truth_token:
            matching_token_count += 1

    total_token_count += 1

return round(matching_token_count / total_token_count * 100, 1)

if __name__ == "__main__":
    path_to_assignment_4 = Path(__file__).parents[0]
    path_to_hs_file = Path(path_to_assignment_4, "icamet-samples_hs.txt")
    path_to_en_file = Path(path_to_assignment_4, "icamet-samples_en.txt")
    path_to_hs_rules_file = Path(path_to_assignment_4, "icamet-samples_hs_rules.txt")

    with open(path_to_hs_file, "r", encoding="utf-8") as hs_file:
        hs_file_contents = hs_file.readlines()[200:]

    with open(path_to_en_file, "r", encoding="utf-8") as en_file:
        en_file_contents = en_file.readlines()[200:]

    with open(path_to_hs_rules_file, "r", encoding="utf-8") as hs_rules_file:
        hs_rules_file_contents = hs_rules_file.readlines()

    print(get_normalization_accuracy(hs_rules_file_contents, en_file_contents))

```